Présentation de AXIOS

YouCode
POWERED BY SIMPLON

Aamir El Amiri

Table des matières

1- Introduction à Axios

2- Pourquoi utiliser Axios au lieu de fetch ?

3- Configuration de base

4-Utilisation d’Axios (GET, POST, etc.)

5- Avantages par rapport à fetch

6-Interceptors

7- Installation d’Axios

8- Cas d’utilisation axios

9 - Conclusion

Introduction

Dans tout projet web, la communication entre le frontend et le backend se fait généralement à travers des requêtes HTTP. Axios est une bibliothèque JavaScript qui facilite énormément cette communication.



Qu’est-ce qu’Axios ?

Axios est une bibliothèque HTTP basée sur les Promises qui permet d’envoyer des requêtes HTTP facilement depuis un navigateur ou un environnement Node.js.



Pourquoi utiliser Axios au lieu de Fetch

Bien que Fetch soit intégré nativement dans les navigateurs modernes, Axios propose des fonctionnalités supplémentaires qui simplifient la gestion des requêtes HTTP

Fetch : l’approche native et minimaliste

💡 Points forts :

- Pas besoin d’installation, c’est inclus nativement dans JavaScript.
- Idéal pour les petits projets ou une utilisation ponctuelle.

⚠️ Limites :

- La gestion des erreurs HTTP (404, 500) n’est pas automatique. Vous devez vérifier manuellement si la réponse est "OK".
- Pas de fonctionnalités avancées comme les intercepteurs ou l’annulation des requêtes.

🎯 Exemple d’utilisation

```
fetch('https://api.example.com/data')
  .then(response => {
    if (!response.ok) {
      throw new Error('Erreur réseau');
    }
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.error('Erreur :', error));
```

Axios : la bibliothèque puissante et pratique

💡 Points forts :

- Simplifie les appels API avec une syntaxe concise.
- Gère automatiquement les erreurs HTTP.
- Fournit des fonctionnalités avancées comme les intercepteurs, la gestion des timeout, et l’annulation des requêtes.

⚠️ Limites :

- Nécessite une installation via npm install axios.
- Peut être "trop" pour les projets simples.

🎯 Exemple d’utilisation

```
import axios from 'axios';

axios.get('https://api.example.com/data')
  .then(response => console.log(response.data))
  .catch(error => console.error('Erreur :', error));
```

Fonctionnalité / Critère	Axios ✅	fetch ❌
✅ **Syntaxe plus simple**	Oui (moins de code)	Plus de verbosité
✅ **Transformation JSON**	Automatique	Nécessite `.json()` manuellement
✅ **Gestion des erreurs**	Plus intuitive (basée sur le code HTTP)	Seules les erreurs réseau sont catchées
✅ **Intercepteurs**	Oui (requêtes et réponses)	Non
✅ **Annulation de requête**	Oui (CancelToken)	Compliqué (via AbortController)
✅ **Support de la configuration globale**	Oui (baseURL, headers, etc.)	Non (manuellement à chaque appel)
✅ **Sérialisation des données**	Automatique pour `params` et `data`	Manuelle
✅ **Timeout**	Intégré	À gérer manuellement avec `AbortController`
✅ **Support JSON natif dans Node.js**	Oui	Non, fetch en Node nécessite un polyfill
✅ **Requêtes en parallèle** (`Promise.all`)	Simplifié avec `axios.spread`	Possible mais plus verbeux

Quand choisir Fetch ou Axios ?

👉 Fetch est parfait si :

- Vous voulez une solution légère et native.
- Vous travaillez sur un projet simple ou de petite envergure.

👉 Axios est idéal si :

- Vous avez besoin de fonctionnalités avancées comme les intercepteurs ou l’annulation des requêtes.
- Votre projet est complexe et nécessite une gestion robuste des appels API.

Les Interceptors dans Axios

```
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

Installation d'Axios

Avec npm :

```
$ npm install axios
```

Avec bower :

```
$ bower install axios
```

Avec yarn :

```
$ yarn add axios
```

Avec le CDN jsDelivr :

```
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
```

Avec le CDN unpkg :

```
<script src="https://unpkg.com/axios/dist/axios.min.js"></script>
```

🎯 Configuration d’Axios

Créer un fichier api.js ou api.ts pour centraliser la configuration :

```
import axios from 'axios';

const api = axios.create({
  baseURL: process.env.REACT_APP_API_URL,
  headers: {
    'Content-Type': 'application/json',
    Authorization: `Bearer ${process.env.REACT_APP_API_TOKEN}`
  },
  timeout: 10000 // 10 secondes
});

export default api;
```

Cas Pratiques d’utilisation



1. Application React avec gestion des tokens JWT
2. Annuler une requête en cas de changement de page
3. Gestion centralisée des erreurs API
4. Chargement de données multiples avec axios.all()

Conclusion

Axios est un outil puissant et flexible qui facilite le développement web moderne. Grâce à ses fonctionnalités avancées, il est fortement recommandé dans les projets React, Node.js, ou tout autre framework nécessitant des appels API.



Et vous ?

Quel outil utilisez-vous pour vos appels API ?